```
12345.to_s(:human)              # => "12.3 Thousand"
1234567.to_s(:human)            # => "1.23 Million"
1234567890.to_s(:human)         # => "1.23 Billion"
1234567890123.to_s(:human)      # => "1.23 Trillion"
1234567890123456.to_s(:human)   # => "1.23 Quadrillion"
```

Defined in `active_support/core_ext/numeric/conversions.rb`.

7 Extensions to `Integer`

7.1 `multiple_of?`

The method `multiple_of?` tests whether an integer is multiple of the argument:

```
2.multiple_of?(1) # => true
1.multiple_of?(2) # => false
```

Defined in `active_support/core_ext/integer/multiple.rb`.

7.2 `ordinal`

The method `ordinal` returns the ordinal suffix string corresponding to the receiver integer:

```
1.ordinal    # => "st"
2.ordinal    # => "nd"
53.ordinal   # => "rd"
2009.ordinal # => "th"
-21.ordinal  # => "st"
-134.ordinal # => "th"
```

Defined in `active_support/core_ext/integer/inflections.rb`.

7.3 `ordinalize`

The method `ordinalize` returns the ordinal string corresponding to the receiver integer. In comparison, note that the `ordinal` method returns **only** the suffix string.

```
1.ordinalize    # => "1st"
2.ordinalize    # => "2nd"
53.ordinalize   # => "53rd"
2009.ordinalize # => "2009th"
-21.ordinalize  # => "-21st"
-134.ordinalize # => "-134th"
```

Defined in `active_support/core_ext/integer/inflections.rb`.

8 Extensions to `BigDecimal`

8.1 `to_s`

The method `to_s` is aliased to `to_formatted_s`. This provides a convenient way to display a BigDecimal value in floating-point notation:

```
BigDecimal.new(5.00, 6).to_s  # => "5.0"
```

8.2 `to_formatted_s`

Te method `to_formatted_s` provides a default specifier of "F". This means that a simple call to `to_formatted_s` or `to_s` will result in floating point representation instead of engineering notation:

```
BigDecimal.new(5.00, 6).to_formatted_s  # => "5.0"
```

and that symbol specifiers are also supported:

```
BigDecimal.new(5.00, 6).to_formatted_s(:db)  # => "5.0"
```

Engineering notation is still supported:

```
BigDecimal.new(5.00, 6).to_formatted_s("e")  # => "0.5E1"
```

9 Extensions to `Enumerable`

9.1 `sum`

The method `sum` adds the elements of an enumerable:

```
[1, 2, 3].sum # => 6
```